Title: CPU Scheduling and Performance Analysis Using FCFS (First-Come, First-Served)

1. AIM

To study and implement the First-Come, First-Served (FCFS) CPU scheduling algorithm in C language and calculate the Average Waiting Time (AWT) and Average Turnaround Time (ATAT) for a given set of processes.

2. THEORY

2.1 Introduction to CPU Scheduling

CPU scheduling is an important function of an operating system that determines which process will be allocated the CPU for execution.

In a multiprogramming operating system, multiple processes may be present in the ready queue. Since the CPU can execute only one process at a time on a single processing core, the operating system uses scheduling algorithms to select the next process for execution.

The main objectives of CPU scheduling are:

1. Maximum CPU utilization.
2. Minimum waiting time.
3. Minimum turnaround time.
4. Maximum throughput.
5. Minimum response time.
6. Fair allocation of CPU resources.

2.2 Introduction to FCFS Scheduling

First-Come, First-Served (FCFS) is the simplest CPU scheduling algorithm.

It follows the FIFO (First-In, First-Out) principle, meaning that the process that arrives first in the ready queue gets executed first.

FCFS is a non-preemptive scheduling algorithm. Once a process gets the CPU, it continues execution until its burst time is completed.

Working of FCFS:

1. Processes enter the ready queue according to their arrival times.

2. The CPU selects the process that arrived first.
3. The selected process executes until completion.
4. The next process in the queue receives the CPU.
5. This process continues until all processes are completed.

2.3 Characteristics of FCFS

- It is a non-preemptive scheduling algorithm.
- It follows FIFO queue management.
- It is simple to implement.
- It is easy to understand.
- Processes are generally executed in arrival order.
- It may result in higher average waiting time.
- It does not consider the burst time of processes when selecting the next process.

2.4 Important Scheduling Terms

1. Arrival Time (AT)

Arrival Time is the time at which a process enters the ready queue and becomes available for CPU execution.

2. Burst Time (BT)

Burst Time is the total CPU execution time required by a process to complete its execution.

3. Completion Time (CT)

Completion Time is the time at which a process finishes its execution.

4. Turnaround Time (TAT)

Turnaround Time is the total time taken by a process from its arrival until its completion.

Formula:

$$TAT = CT - AT$$

5. Waiting Time (WT)

Waiting Time is the total time a process spends waiting in the ready queue before getting CPU execution.

Formula:

$$WT = TAT - BT$$

6. Average Waiting Time (AWT)

Average Waiting Time is the average waiting time of all processes.

Formula:

$$AWT = \frac{\sum WT}{n}$$

Where:

- WT = Waiting Time.
- n = Total number of processes.

7. Average Turnaround Time (ATAT)

Average Turnaround Time is the average turnaround time of all processes.

Formula:

$$ATAT = \frac{\sum TAT}{n}$$

2.5 Gantt Chart

A Gantt chart is a graphical representation of process execution over time.

It shows:

- The order of process execution.
- Starting time of each process.
- Completion time of each process.
- Total CPU execution timeline.

Example:

```
None
0        5        8        12

|   P1   |   P2   |   P3   |
```

In this example:

- P1 executes from 0 to 5.
- P2 executes from 5 to 8.
- P3 executes from 8 to 12.

2.6 Advantages of FCFS

1. Simple and easy to implement.
2. Requires minimal scheduling overhead.
3. Fair according to arrival order.
4. No process starvation due to scheduling order.
5. Suitable for simple batch-processing systems.

2.7 Disadvantages of FCFS

1. Long processes can make short processes wait for a long time.
2. Average waiting time can be high.
3. It does not prioritize shorter processes.
4. It may suffer from the Convoy Effect.
5. It is not suitable for interactive time-sharing systems.

2.8 Convoy Effect

The Convoy Effect occurs when several short processes are forced to wait behind one long process.

For example, if a process requires 20 milliseconds of CPU time and several other processes require only 2 milliseconds each, the shorter processes must wait until the longer process completes.

This increases the average waiting time and reduces system performance.

2.9 Example of FCFS Scheduling

Consider the following processes:

Process	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	4

Gantt Chart:

```
None
0             5         8              12

|     P1      |  P2     |      P3       |
```

Calculation Table:

Process	AT	BT	CT	TAT	WT
P1	0	5	5	5	0
P2	1	3	8	7	4
P3	2	4	12	10	6

Turnaround Time:

- P1 = 5 − 0 = 5
- P2 = 8 − 1 = 7

- P3 = 12 − 2 = 10

Waiting Time:

- P1 = 5 − 5 = 0
- P2 = 7 − 3 = 4
- P3 = 10 − 4 = 6

Average Waiting Time:

$$AWT = \frac{0+4+6}{3}$$

$$AWT = 3.33$$

Average Turnaround Time:

$$ATAT = \frac{5+7+10}{3}$$

$$ATAT = 7.33$$

3. CODE

Program: Implementation of FCFS CPU Scheduling Algorithm in C

Filename: `fcfs.c`

```
None
#include <stdio.h>


int main()

{
```

```
int n, i, j;


int at[20], bt[20];

int ct[20], tat[20], wt[20];

int pid[20];


float avg_wt = 0;

float avg_tat = 0;


printf("Enter number of processes: ");

scanf("%d", &n);


if (n <= 0 || n > 20)

{

    printf("Invalid number of processes.\n");

    return 1;

}


// Input process details

for (i = 0; i < n; i++)

{
```

```
        pid[i] = i + 1;

        printf("\nEnter Arrival Time of P%d: ", pid[i]);
        scanf("%d", &at[i]);

        printf("Enter Burst Time of P%d: ", pid[i]);
        scanf("%d", &bt[i]);

        if (at[i] < 0 || bt[i] <= 0)
        {
            printf("Invalid arrival or burst time.\n");
            return 1;
        }
    }

    // Sort processes according to arrival time
    for (i = 0; i < n - 1; i++)
    {
        for (j = 0; j < n - i - 1; j++)
        {
            if (at[j] > at[j + 1])
```

```
        {
            int temp;

            temp = at[j];
            at[j] = at[j + 1];
            at[j + 1] = temp;

            temp = bt[j];
            bt[j] = bt[j + 1];
            bt[j + 1] = temp;

            temp = pid[j];
            pid[j] = pid[j + 1];
            pid[j + 1] = temp;
        }
    }
}

// Calculate completion time
int current_time = 0;
```

```
for (i = 0; i < n; i++)
{
    if (current_time < at[i])
    {
        current_time = at[i];
    }

    current_time = current_time + bt[i];

    ct[i] = current_time;

    // Calculate turnaround time
    tat[i] = ct[i] - at[i];

    // Calculate waiting time
    wt[i] = tat[i] - bt[i];

    avg_wt += wt[i];
    avg_tat += tat[i];
}
```

```
// Calculate averages
avg_wt = avg_wt / n;
avg_tat = avg_tat / n;

// Display process details
printf("\nFCFS Scheduling Results\n");

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

for (i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
           pid[i], at[i], bt[i],
           ct[i], tat[i], wt[i]);
}

// Display average times
printf("\nAverage Waiting Time = %.2f", avg_wt);

printf("\nAverage Turnaround Time = %.2f\n", avg_tat);
```

```
// Display Gantt chart
printf("\nGantt Chart:\n");

printf(" ");

for (i = 0; i < n; i++)
{
    printf("--------");
}

printf("\n|");

for (i = 0; i < n; i++)
{
    printf("  P%d   |", pid[i]);
}

printf("\n ");

for (i = 0; i < n; i++)
{
```

```
        printf("--------");
    }

    printf("\n");

    return 0;
}
```

4. EXECUTION INSTRUCTIONS

Requirements

- Operating System: Ubuntu Linux / Kali Linux.
- Programming Language: C.
- Compiler: GCC.
- Terminal: Linux Terminal.

Step 1: Open Terminal

Press `Ctrl + Alt + T` to open the Linux terminal.

Step 2: Check GCC Compiler

```
None
gcc --version
```

If GCC is not installed:

```
None
sudo apt update

sudo apt install build-essential
```

Step 3: Create the Program File

```
None

nano fcfs.c
```

Copy and paste the complete C program into the editor.

Save the file using:

- Press `Ctrl + O`.
- Press Enter.
- Press `Ctrl + X`.

Step 4: Compile the Program

```
None

gcc fcfs.c -o fcfs
```

Step 5: Execute the Program

```
None

./fcfs
```

Step 6: Enter Input

Enter the following sample input:

```
None
Enter number of processes: 3


Enter Arrival Time of P1: 0

Enter Burst Time of P1: 5


Enter Arrival Time of P2: 1
```

```
Enter Burst Time of P2: 3


Enter Arrival Time of P3: 2

Enter Burst Time of P3: 4
```

Step 7: Observe the Output

Expected Output:

```
None
FCFS Scheduling Results


PID     AT      BT      CT      TAT     WT

P1      0       5       5       5       0

P2      1       3       8       7       4

P3      2       4       12      10      6


Average Waiting Time = 3.33

Average Turnaround Time = 7.33


Gantt Chart:


 ---------------------------------

|  P1   |  P2   |  P3   |
```

```
--------------------------------
```

5. RESULT

The First-Come, First-Served (FCFS) CPU scheduling algorithm was successfully implemented using the C programming language.

The program calculated the completion time, turnaround time and waiting time for each process.

For the given sample input:

- Average Waiting Time = 3.33 time units.
- Average Turnaround Time = 7.33 time units.

The Gantt chart was also generated to represent the process execution order.

6. CONCLUSION

In this practical, the First-Come, First-Served CPU scheduling algorithm was studied and implemented using C programming. Processes were scheduled according to their arrival times using the FIFO principle. Completion time, waiting time and turnaround time were calculated for each process. The average waiting time and average turnaround time were also determined. Thus, the practical helped in understanding non-preemptive CPU scheduling and its performance analysis.